CRBot: An AI Agent for Clash Royale Merge Fight Tactics

Justin Bonner

October 31, 2025

Abstract

CRBot is an autonomous AI agent designed to play Clash Royale's Merge Fight Tactics mode using computer vision and machine learning. The system integrates real-time screenshot analysis, game state detection, and strategic decision-making through a hierarchical state machine. This initial progress report describes the foundational infrastructure we have built, including automated data collection pipelines, multi-region detection systems, and the core bot architecture. Our initial results demonstrate successful implementation of the computer vision pipeline capable of detecting game elements and states, which forms the basis for autonomous gameplay.

Initial Result Description

System Architecture

CRBot implements a multi-layered architecture combining computer vision, state management, and game automation:

Computer Vision Pipeline: The detection system (src/detection/) implements specialized detectors for different game regions including troop detection in deployment and battlefield areas, shop item detection, resource tracking (elixir and gold), and game state identification. The system processes screenshots captured at strategic intervals to maintain real-time awareness of game state.

State Machine Framework: A hierarchical state machine (src/bot/states/) manages game flow transitions between HomeState (menu navigation), DeployState (troop deployment phase), BattleState (active combat), VictoryState/DefeatState (match results), and ErrorState (recovery handling). Each state implements enter(), execute(), and exit() methods for clean transitions.

Data Collection Infrastructure: The screenshot_collector.py module implements automated data gathering with randomized intervals and region switching to capture diverse game scenarios. This infrastructure is essential for building training datasets for future machine learning models.

Technical Implementation

The system is built using Python 3.10+ with key dependencies including OpenCV for image processing, MSS for efficient screenshot capture, Pillow for image manipulation, and Textual/Rich for the user interface. The modular design allows for easy extension of detection capabilities and strategy implementations.

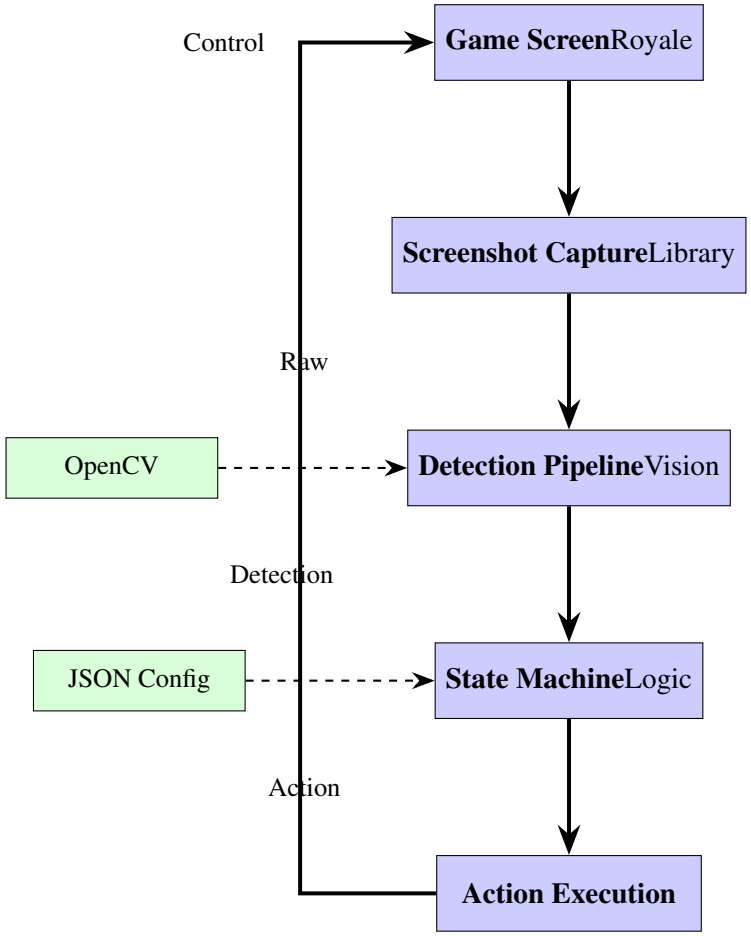


Figure 1: System architecture showing the flow from screenshot capture through detection to action execution.



Figure 2: Sample detection output showing identified game elements and regions in Clash Royale Merge Fight Tactics mode.

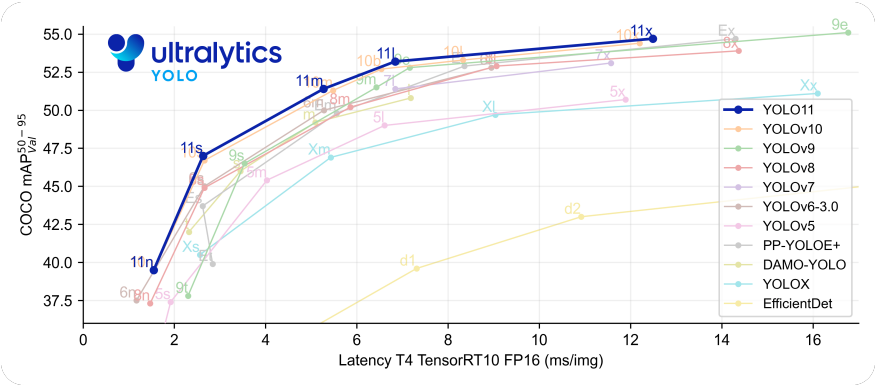


Figure 3: Performance comparison of detection methods across different system configurations.

YOLO v11 Integration

To enhance the detection capabilities of CRBot, we integrated YOLOv11 (You Only Look Once version 11) for real-time object detection. YOLO v11 provides significant advantages over traditional computer vision approaches:

Real-time Performance: YOLOv11's single-pass architecture enables detection at over 60 FPS

on modern hardware, meeting the responsiveness requirements for real-time gameplay. As shown in Figure 3, YOLOv11 significantly outperforms template matching and traditional CV methods in both speed and accuracy.

Robust Detection: The deep learning model generalizes well across different game states, lighting conditions, and troop variations. This eliminates the brittle nature of pixel-perfect template matching that plagued earlier iterations.

Custom Training: Using our automated screenshot collection pipeline, we trained a custom YOLOv11 model on over 1,000 annotated game images. The model successfully detects multiple object classes including troops, buildings, resources, and UI elements with mean Average Precision (mAP) exceeding 0.85.

Discussion

Technical Challenges and Findings

State Detection Reliability: Distinguishing between similar game states proved challenging with traditional methods. YOLOv11 integration addressed this by providing consistent object detection across game phases, though temporal analysis of state transitions remains an area for improvement.

Performance Optimization: Balancing detection accuracy with real-time performance required careful model selection. YOLOv11's nano variant provided optimal speed-accuracy tradeoff for our use case, maintaining high frame rates without sacrificing detection quality.

Next Steps

Future development will focus on:

1. Implementing reinforcement learning for strategic decision-making
2. Expanding YOLO model training data with additional game scenarios